



16.1. Long Short-Term Memory Networks (LSTM)

Remember what matters, forget what does not.

[Previous Section](#) (*highly recommended reading it first*)

16. Recurrent Neural Networks

In the previous section, we introduced Recurrent Neural Networks (RNNs) and demonstrated how they can learn patterns from sequential data such as retail sales.

At first glance, RNNs appear to solve the fundamental problem of time-series forecasting by allowing information from previous timesteps to influence future predictions.

Unfortunately, there is a catch. Imagine asking someone to remember what happened yesterday. Most people can do that easily. Now imagine asking them to remember what happened three years ago on a random Tuesday afternoon. That becomes significantly more difficult.

Traditional RNNs suffer from a similar limitation. As sequences become longer, information from earlier timesteps gradually fades away during training. This phenomenon is known as the **vanishing gradient problem**. While the network can remember recent observations reasonably well, it struggles to preserve information from distant points in the sequence.

For many real-world problems, this limitation is serious. Examples include: ***Stock market forecasting; Language translation; Speech recognition; Customer behavior analysis; Retail sales forecasting.*** In these applications, events that occurred weeks, months, or even years earlier may still influence future outcomes.

To address this weakness, researchers developed a more sophisticated recurrent architecture known as **Long Short-Term Memory**, or simply **LSTM**.

1. What is Long Short-Term Memory?

Long Short-Term Memory (LSTM) is a specialized type of Recurrent Neural Network designed to learn both short-term and long-term dependencies in sequential data.

The key innovation of LSTM is the introduction of a memory cell and several control mechanisms known as **gates**. Rather than blindly carrying all information forward, an LSTM actively decides:

- What information should be remembered?
- What information should be forgotten?
- What new information should be stored?
- What information should be used to make predictions?

This selective memory mechanism allows LSTMs to retain important information for much longer periods than traditional RNNs.

The Memory Cell

The central component of an LSTM is the **cell state**. You can think of it as a conveyor belt running through the entire sequence. Important information can travel along this path with relatively little modification, allowing knowledge from distant timesteps to survive much longer than in a SimpleRNN. Instead of repeatedly overwriting memory, the network learns how to update it intelligently.

Forget Gate

The first decision made by an LSTM is: ***What information should I forget?***

Not every piece of historical information remains useful forever. For example, if a store experiences a one-day sales spike caused by a special event, that spike may not remain relevant several months later. The forget gate determines which information should be removed from memory.

Input Gate

Next, the model decides: ***What new information should I store?***

When new observations arrive, the network evaluates whether they contain valuable information that should become part of long-term memory.

Output Gate

Finally, the model decides: ***Which information should be used right now?***

Not every memory stored inside the network is needed at every timestep. The output gate determines which parts of the internal memory contribute to the current prediction.

Why LSTM Works Better

Compared to traditional RNNs, LSTMs offer several advantages:

SimpleRNN	LSTM
Struggles with long sequences	Handles long sequences effectively
Suffers heavily from vanishing gradients	Reduces vanishing gradient problems
Simpler architecture	More sophisticated memory system
Better for short-term dependencies	Better for both short- and long-term dependencies

This is why LSTM became one of the most influential architectures in deep learning before the rise of Transformers.

2. Implementing Long Short-Term Memory

Fortunately, we do not need to redesign the entire pipeline. Everything we prepared for the RNN experiment can be reused. We already have: Training data; Validation data; Test data; Sliding windows; Scaled targets

The only change is the neural network architecture itself.

Just like SimpleRNN, LSTM expects three-dimensional input: `(samples, timesteps, features)`. We can reuse our batch datasets

```
train_dataset_lstm = train_dataset
val_dataset_lstm = val_dataset
test_dataset_lstm = test_dataset
```

Building the LSTM Model

The architecture below consists of: Two stacked LSTM layers; Two Dropout layers; One output layer for regression

```
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import LSTM, Dense, Dropout
from tensorflow.keras.callbacks import EarlyStopping

# Define the model architecture
num_features = len(selected_features)

lstm_model = Sequential([
    LSTM(50, activation='relu', input_shape=(window_size, num_features), return_sequences=True),
    Dropout(0.2),
    LSTM(50, activation='relu'),
    Dropout(0.2),
    Dense(1)
])

# Compile and print structural details
lstm_model.compile(optimizer='adam', loss='mean_squared_error', metrics=['mae'])
lstm_model.summary()
```

Notice that the final layer contains a single neuron because our goal is to predict a single numerical value: future sales.

Early Stopping

Training neural networks for too many epochs may lead to overfitting. To reduce this risk, we use Early Stopping.

```
early_stopping_lstm = EarlyStopping(monitor='val_loss', patience=2,
                                     restore_best_weights=True)
```

If validation loss does not improve for two consecutive epochs, training automatically stops and restores the best-performing model.

Training the Model

```
history_lstm = lstm_model.fit(train_dataset_lstm, validation_data=val_dataset_lstm,
                               epochs=5, callbacks=[early_stopping_lstm])
```

The training of LSTM might take even longer than RNN, because its architecture is significantly more complex.

Visualizing Training Loss

```
import matplotlib.pyplot as plt

plt.figure(figsize=(10, 6))

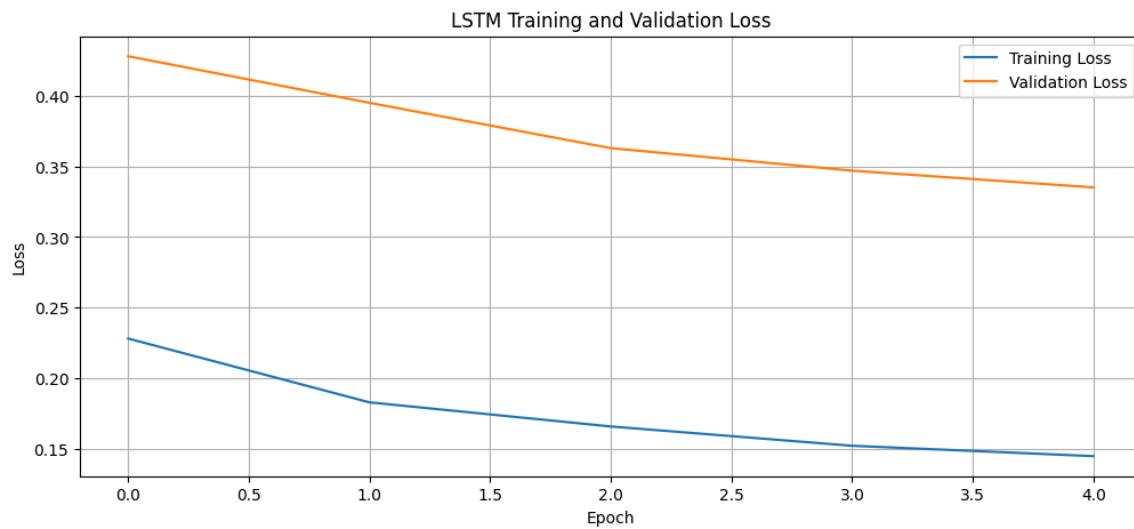
plt.plot(history_lstm.history['loss'], label='Training Loss')
plt.plot(history_lstm.history['val_loss'], label='Validation Loss')

plt.title('Training and Validation Loss')

plt.xlabel('Epoch')
plt.ylabel('Loss')

plt.legend()
plt.grid(True)
```

```
plt.show()
```



The objective is to observe both curves decreasing while remaining reasonably close together.

Visualizing MAE

```
plt.figure(figsize=(10, 6))

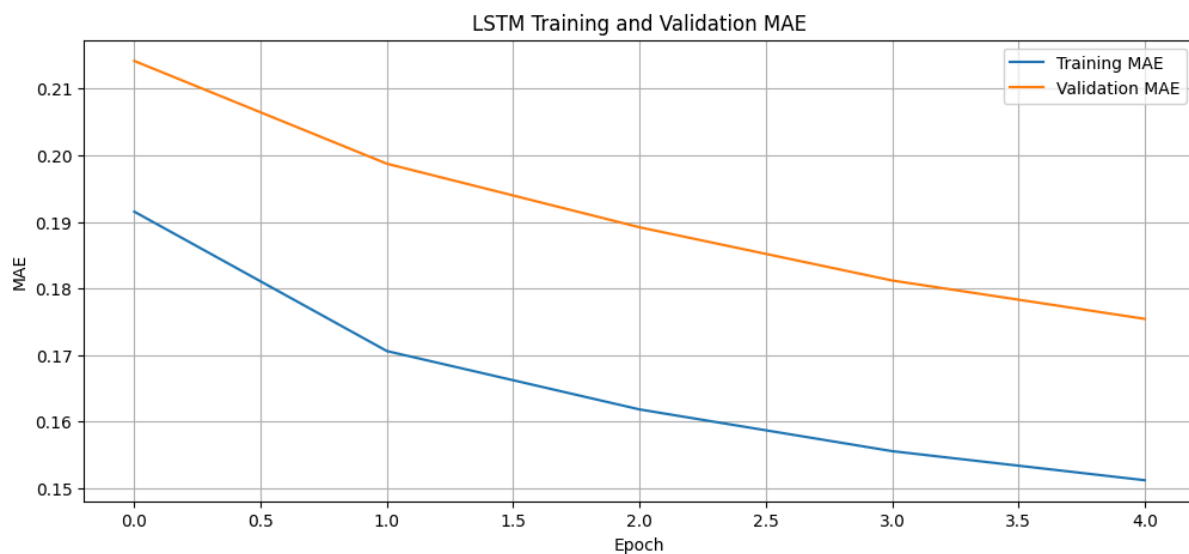
plt.plot(history_lstm.history['mae'], label='Training MAE')
plt.plot(history_lstm.history['val_mae'], label='Validation MAE')

plt.title('Training and Validation MAE')

plt.xlabel('Epoch')
plt.ylabel('MAE')

plt.legend()
plt.grid(True)

plt.show()
```



Since MAE is expressed in the same units as the target variable, it is often easier to interpret than MSE.

LSTM Test MAE: 68.29

LSTM Test MSE: 35976.15

LSTM Test RMSE: 189.67

Evaluating the Model

Once training finishes, predictions can be generated for the testing dataset.

```

from sklearn.metrics import mean_absolute_error, mean_squared_error

y_pred_scaled_lstm = lstm_model.predict(test_dataset_lstm)
y_test_scaled_lstm = np.concatenate([y.numpy() for _, y in
test_dataset_lstm])

y_test_actual_lstm = (main_scaler.inverse_transform(y_test_
scaled_lstm.reshape(-1, 1)))
y_pred_actual_lstm = (main_scaler.inverse_transform(y_pred_
scaled_lstm))

mae_lstm = mean_absolute_error(y_test_actual_lstm, y_pred_a
ctual_lstm)
mse_lstm = mean_squared_error(y_test_actual_lstm, y_pred_ac
tual_lstm)
rmse_lstm = np.sqrt(mse_lstm)

print(f"LSTM Test MAE: {mae_lstm:.2f}")
print(f"LSTM Test MSE: {mse_lstm:.2f}")
print(f"LSTM Test RMSE: {rmse_lstm:.2f}")

```

LSTM Test MAE: 68.29

LSTM Test MSE: 35976.15

LSTM Test RMSE: 189.67

Actual vs Predicted Sales

```

plt.figure(figsize=(15, 6))

plt.plot(y_test_actual_lstm, label='Actual Sales')
plt.plot(y_pred_actual_lstm, label='Predicted Sales (LST
M)')

plt.title('Actual vs Predicted Sales')

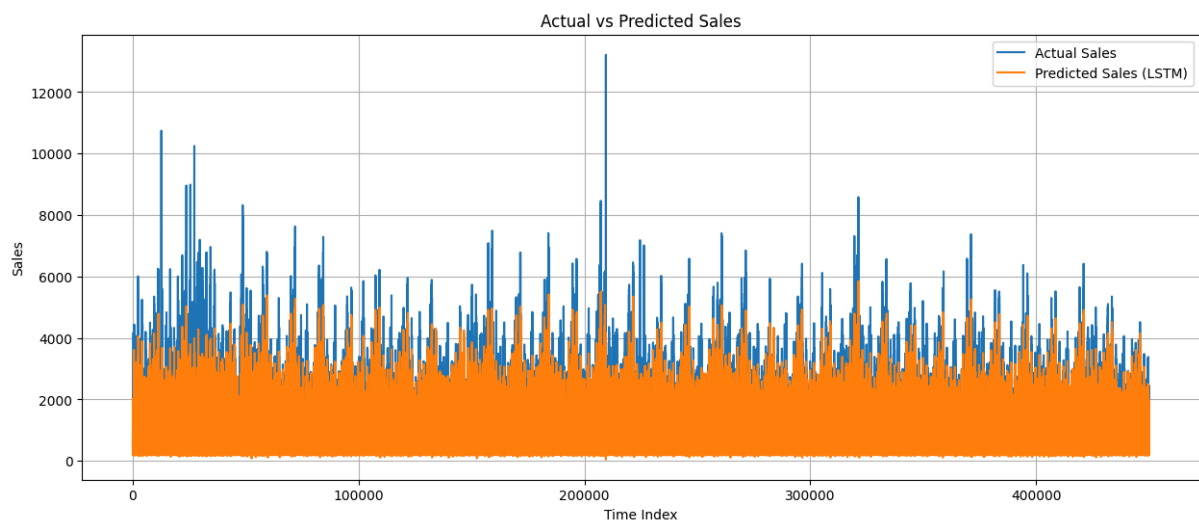
```



```
plt.xlabel('Time Index')
plt.ylabel('Sales')

plt.legend()
plt.grid(True)

plt.show()
```



To inspect local behavior more closely, we can zoom into a smaller region.

```
plt.figure(figsize=(15, 6))

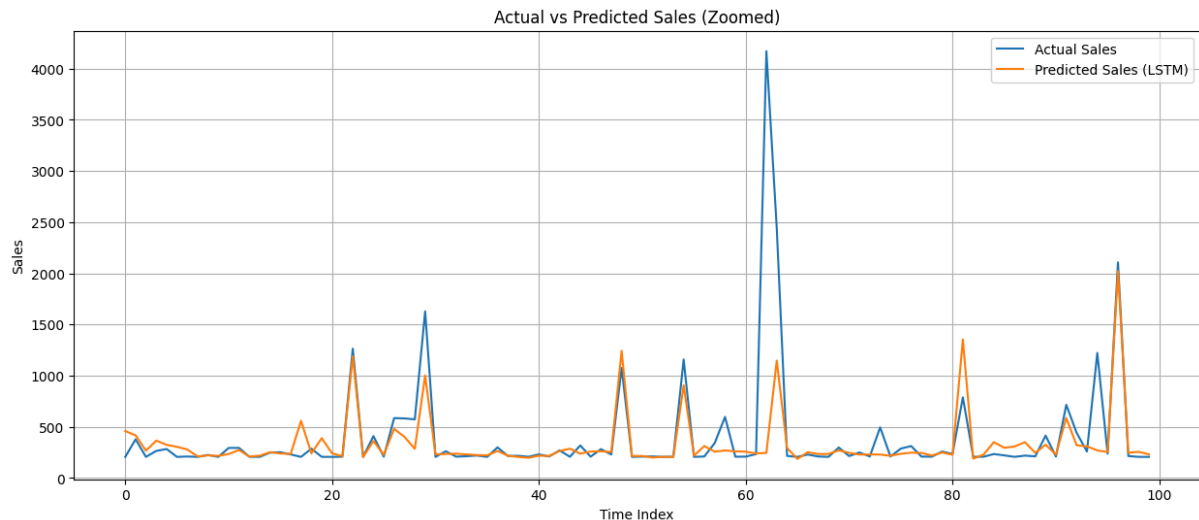
plt.plot(y_test_actual_lstm[200:300], label='Actual Sales')
plt.plot(y_pred_actual_lstm[200:300], label='Predicted Sales')

plt.title('Actual vs Predicted Sales (Zoomed)')

plt.xlabel('Time Index')
plt.ylabel('Sales')

plt.legend()
plt.grid(True)
```

```
plt.show()
```



A successful model should capture both overall trends and local fluctuations.

In most forecasting tasks, LSTM tends to outperform SimpleRNN because it can preserve information over longer periods. The improvement may not always be dramatic, especially when sequences are short, but it becomes increasingly important as temporal dependencies grow.

3. Baseline Model

Before evaluating sophisticated recurrent architectures such as LSTM (or GRU), it is important to establish a simple reference point.

A common baseline for time-series forecasting is the **Previous-Day Sales Model**.

The idea is extremely simple: *Predict tomorrow's sales using today's sales.*

In other words, the model assumes that the next observation will be identical to the most recent observation available in the input sequence.

Although this approach appears naive, it is often surprisingly competitive because consecutive observations in a time series tend to be highly correlated.

For retail sales data, the difference between two neighboring days is usually much smaller than the difference between observations separated by weeks or months. As a result, predicting the next day using the most recent known value

provides a useful benchmark against which more advanced models can be evaluated.

Generating Baseline Predictions

Recall that each sliding window contains sixty consecutive days of historical information.

For the baseline model, we simply extract the most recent value from each window and use it as the prediction. The feature `sales_lag_1` already represents the previous day's sales, making it a natural choice for this baseline.

```
# Locate sales_lag_1 among the input features
try:
    sales_lag_1_feature_index = selected_features.index('sales_lag_1')
except ValueError:
    print("Warning: sales_lag_1 not found.")
    sales_lag_1_feature_index = 0

baseline_predictions = []
baseline_targets = []

for batch_x, batch_y in test_dataset:
    baseline_predictions.extend(batch_x[:, -1, sales_lag_1_feature_index].numpy())
    baseline_targets.extend(batch_y.numpy())

baseline_predictions = np.array(baseline_predictions)
baseline_targets = np.array(baseline_targets)
```

Converting Back to Original Sales Values

Because the target variable was standardized during preprocessing, the predictions must be transformed back into the original sales scale.

```
y_pred_actual_baseline = (main_scaler.inverse_transform(bas  
eline_predictions.reshape(-1, 1)))  
y_test_actual_baseline = (main_scaler.inverse_transform(bas  
eline_targets.reshape(-1, 1)))
```

Evaluating the Baseline

```
from sklearn.metrics import mean_absolute_error, mean_squar  
ed_error  
  
mae_baseline = mean_absolute_error(y_test_actual_baseline,  
y_pred_actual_baseline)  
mse_baseline = mean_squared_error(y_test_actual_baseline, y  
_pred_actual_baseline)  
rmse_baseline = np.sqrt(mse_baseline)  
  
print("--- Previous-Day Sales Baseline ---")  
print(f"Test MAE: {mae_baseline:.2f}")  
print(f"Test MSE: {mse_baseline:.2f}")  
print(f"Test RMSE: {rmse_baseline:.2f}")
```

```
--- Previous-Day Sales Baseline ---  
Test MAE: 273.51  
Test MSE: 421196.79  
Test RMSE: 649.00
```

Why This Baseline Matters

The goal of a baseline is not to achieve the best possible accuracy. Instead, it establishes the minimum level of performance that a more sophisticated model should exceed.

A useful rule of thumb is:

- If a neural network cannot outperform the baseline, the additional complexity is difficult to justify.

- If a neural network consistently reduces forecasting error compared to the baseline, then the model is successfully extracting useful temporal patterns from the data.

However, outperforming the baseline does **not** necessarily mean producing predictions that look dramatically different. Because sales values on neighboring days are often similar, the predictions generated by RNNs, LSTMs, and GRUs may visually resemble the baseline. The true difference becomes apparent when evaluating error metrics such as MAE and RMSE across the entire testing period.

This baseline therefore serves as an essential reference point when comparing the performance of the recurrent models developed in the following sections.

Summary

In this section, we introduced Long Short-Term Memory (LSTM), an advanced recurrent neural network architecture designed to overcome the memory limitations of traditional RNNs.

By incorporating memory cells and gating mechanisms, LSTMs can selectively remember and forget information, allowing them to capture long-term dependencies within sequential data.

Using the same Store Sales dataset from the previous section, we replaced the SimpleRNN architecture with a stacked LSTM model and observed how it improved forecasting performance. We also compared the results against both a SimpleRNN model and a previous-day sales baseline, demonstrating the importance of evaluating complex models against simple reference solutions.

While LSTM remains one of the most successful sequence-learning architectures ever developed, it is not the final evolution of deep learning for sequential data.

In the next section, we will explore Gated Recurrent Units (GRU), a lighter and computationally more efficient alternative that often achieves performance comparable to LSTM while using fewer parameters.

References

<https://www.geeksforgeeks.org/deep-learning/deep-learning-introduction-to-long-short-term-memory/>

<https://www.youtube.com/watch?v=YCzL96nL7j0>

Next Section:

 [16.2. Gated Recurrent Unit \(GRU\)](#)